application? Using WebHooks, you can share data among different applications to reduce, if not eliminate, redundancy.

Extendibility: WebHooks let your users extend your applications to build plug-ins on their own. All you have to do is just specify the call back URL.

And all this can be achieved with the new design that allows users to specify their callback URLs in the application. The application can then return the data to that URL. This way, the calling user does not require access to the application's source code. It's more like an API but in an API call, the user has to invoke the request explicitly, and if there is a WebHook associated with the application, then the callback URL gets notified automatically without having to request the API.

You might find some examples in your daily routine when you use certain applications that seem to work like a WebHook. But they are not the real-time examples. Also, the scenario mentioned above is just a simple example. The practical requirement may vary and this is what triggers the necessity to start using WebHooks.

How do WebHooks work?

A consumer of a Web service or a user is provided with URLs for different events, to which an application will post data when the events occur. Here's an example: on receiving an e-mail, the mail program will be clever enough to post the new message to some URL (*http://mydomain.in/webhooks.php*). So you can avoid repeatedly checking your inbox or pressing the send/receive button of your mail client. The event gets fired as soon as the mail is received. Now it's up to you how to utilise that event. You can keep SMS alerts on that event, or you can write a script to perform some other operation when you receive that e-mail.

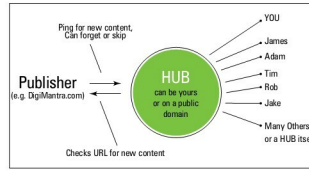
An example to get you started

PubSubHubBub

Have you heard the term PubSubHubBub? I am sure many of you have not. In simple words, it's a new kind of RSS and a step closer to the programmable Web. Some of you might wonder why we need a new RSS? Allow me to dig a little deeper.

How RSS works

RSS, as we all know, is content syndication and is served in the form of a XML file to a RSS reader. Programmers and many Web applications use these feeds to drive many powerful applications like blog aggregators and mashing services. But the problem with RSS is the repeated polling. Polling is a process in which a client pings the server to check for new content. RSS clients check servers for new content after a regular interval of time, set by the user. The server has to respond to every request saying, "You have got the latest content." You can change the ping intervals to the server, but there is no ideal time for it. If you set it to one minute, then the server is pinged every minute, causing overheads to



both the server and the requester. And if the delay is of one hour or more, then it's too late. And it is impossible to set the time correctly, even with an adaptive rate control. However PubSubHubBub tries to eliminate this problem and updates you with the latest content within seconds of the server getting updated. In a scenario like this, even a minute's delay seems to be a lot.

How does PubSubHubBub work?

PubSubHubBub tries to achieve a programmable Web. Instead of repeated polling, in the normal RSS, a URL declares its hub server(s) using the *<link rel= "hub" />* tag. The hubs can be on any public domain or even on your personal server.

After subscribing, if the atom file of a RSS link declares its hub(s), the user can then avoid repeated polling of the URL and can instead register with the feed's hub(s) and subscribe to updates.

Now, the moment new content is published, the publisher pings and notifies the hub about the update. The hub fetches the updated content and broadcasts it to all the clients/users who are subscribed to it. It is more like a push protocol similar to the push e-mail service from RIM's Blackberry. Thus, the subscriber to that hub gets the new content automatically without having to ask for it, saving a lot of resources.

What is wrong with a delay of a minute or an hour?

Since feed updates are important sources of information, in the world of the real-time Web, even a few seconds sounds too long. Real-time applications are something that are achieved as part of a chain of applications. Google Wave is a very good example of the programmable Web. Many mailing services like Mailchimp have already started supporting WebHooks in their APIs. Twitter is also going to support this sometime soon.

Implementation

The implementation of PuBSubHubBub requires the following entities:

1) Publishers
2) Hub(s)
3) Subscribers

Publishers: A publisher is the one responsible for